SWEET1NE

QR-Powered Restaurant Menu, Intelligence & Marketing Platform

Solution Design Document

Version 1.0

Prepared by	SpectreTech Limited
Prepared for	Developer & GlobalSolutions
Client project	Sweet1ne — Chingford & Lewisham
Project reference	GS-SW1-2026-001
Purpose	Solution Design, Scope, Architecture & Delivery Plan
Technical lead	Kelvin Kibiru — Lead Software Developer
Developer	Stephen Nyansoho Machera
Programme duration	18 weeks (target)
Date	July 2026

Document Purpose

This Solution Design Document (SDD) is the single reference that defines **what** the Sweet1ne platform will do, **how** it will be built, and **in what order** it will be delivered. It consolidates the agreed scope, the technology stack, the system architecture, the data model, the user workflows, and the phased delivery plan into one document.

1. Overview

Sweet1ne is a two-branch restaurant (Chingford and Lewisham) with a “soul meets food” brand identity. The platform replaces physical menus and manual insight-gathering with a connected digital system delivered in two releases:

- **Version 1 — Visual Menu Platform:** a mobile-first, QR-accessed digital menu with per-branch content, order building, a staff dashboard, and per-branch analytics capture.
- **Version 2 — Centralised Intelligence, Website & Marketing System:** a public Sweet1ne website that embeds the menu, plus lead capture, a consented contact database, branded email marketing, and an insight layer that turns captured data into ready-made decisions.

> **Guiding principle.** The platform should not simply hand Sweet1ne more data — it should collect it, interpret it, and present it as ready-made answers with a clear next action attached.

1.1 Objectives

- Replace printed menus with a contactless, QR-based digital menu requiring no app install or customer login.
- Give each branch its own menu, QR codes, and analytics while sharing one management system.
- Let staff manage menu content and orders in real time through role-based dashboards.
- Capture engagement data (scans, views, searches, orders) as the foundation for insight.
- In V2, unify menu, website, and marketing data and surface calibrated, actionable recommendations.
- Position the analytics layer as an operational advantage for owners who do not have time to read dashboards.

2. Scope

2.1 In Scope

Version 1 — Visual Menu Platform

- Mobile-first QR visual menu
- Branch-specific menus and QR codes
- Dish imagery, descriptions, prices, dietary and allergen tags
- Search and filtering
- Order building and submission
- Configurable role-based staff dashboard
- Real-time menu control (availability, pricing, content)
- Per-branch analytics for scans, views, searches, and orders

Version 2 — Centralised Intelligence, Website & Marketing System

- New mobile-first Sweet1ne website
- Native embedding of the V1 menu (one-source content)
- Automatic lead capture through website and QR journeys
- Central contact database with consent, source, and branch tagging
- Branded bulk email marketing with editable templates and personalisation
- Campaign delivery and performance measurement
- Integration of menu/QR, website, Google Analytics, and Meta data
- The **Collect → Analyse → Critique → Alert → Act** recommendation layer
- Calibrated thresholds and a weekly insight brief

2.2 Out of Scope

The following are excluded unless added by written Change Request:

- Card or in-app payment processing
- POS or delivery-platform integration
- Loyalty schemes and complex promotions
- Augmented-reality menu previews
- Multi-language support
- Native mobile apps
- Paid media management and ongoing campaign operation
- Ongoing copywriting or photography
- 24-hour support and infrastructure management after handover
- Third-party subscription or usage charges

2.3 Client Dependencies

Delivery timing assumes the client provides, on schedule:

When	What is required
Week 1	Pilot branch; 10–15 starter dishes; approved prices, descriptions, dietary and allergen data; professional photography; baseline sales data; named menu owner.
By Week 8	Google Analytics and Meta access; domain and DNS access; brand assets; approved website content; email standards examples; a working session to approve metrics and thresholds.
Before live lead capture	Approved privacy notice, cookie/tracking notice, marketing wording, retention choices, named privacy contact.
Before production deployment	Approved staff roles, sender-domain configuration, booking attribution method, provider terms.

Client-caused delay extends the programme by at least the period of delay.

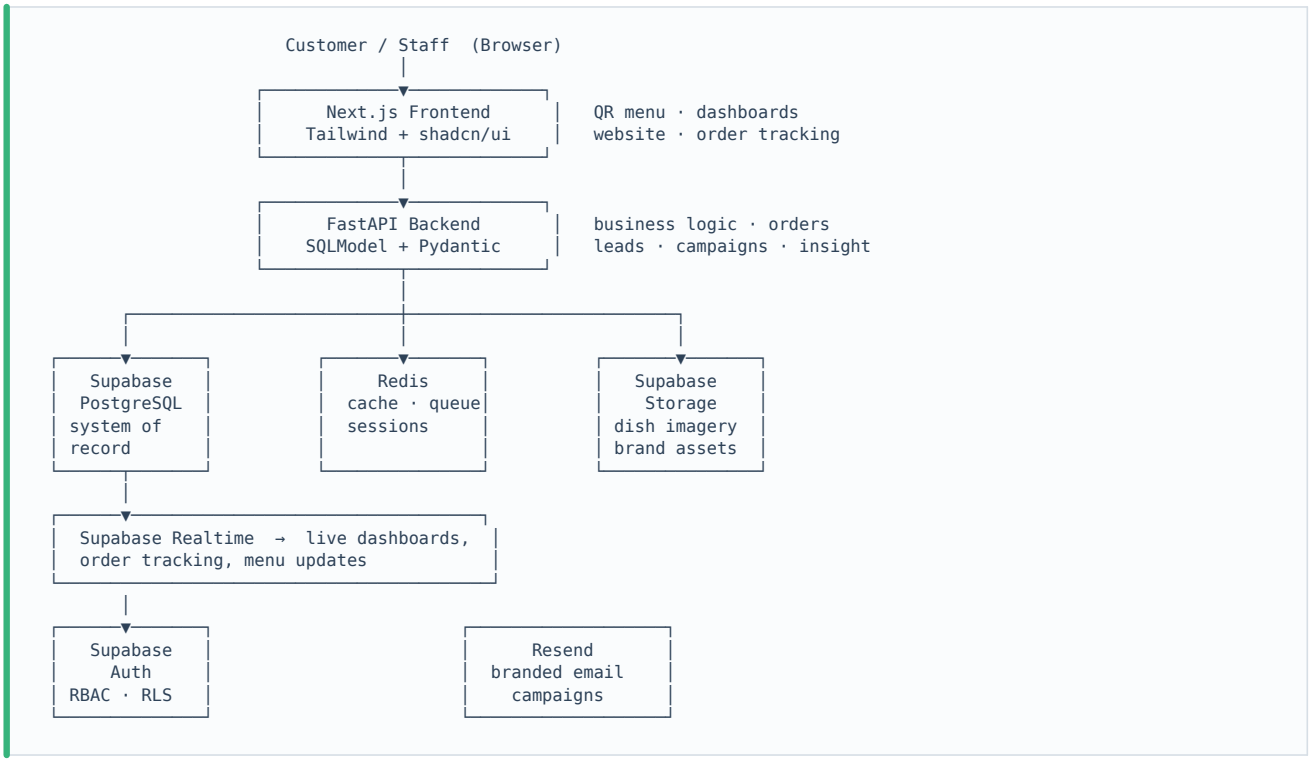
3. Architecture

3.1 Design Approach

The platform uses a centralised FastAPI backend with Supabase PostgreSQL as the relational system of record, and Supabase Realtime for live propagation of order status, menu availability, and dashboard updates. Restaurant data is highly relational — branches, tables, menus, orders, order items, leads, campaigns, and analytics all interconnect — so a relational database with JOINs, foreign keys, and transactions gives reliable reporting and strong consistency, while Supabase's subscription channels keep the experience live.

Redis sits alongside the database for active order state, session data, frequently accessed menus, cached analytics, and rate limiting.

3.2 Layered Architecture



3.3 Layer Responsibilities

Layer	Responsibility
Frontend **Next.js**	Public QR menu, order tracking, staff and admin dashboards, and the V2 website. Server-side rendering for fast menu loads; mobile-first throughout.
Backend **FastAPI**	Centralised business logic: order transitions, menu management, lead capture, campaign orchestration, analytics aggregation, and insight generation.
Database **Supabase PostgreSQL**	System of record for all relational data. Strong consistency, JOINs, transactions, and foreign keys for reliable reporting.
Realtime **Supabase Realtime**	Live propagation of order status, menu availability, and dashboard updates.
Auth **Supabase Auth**	Staff authentication, role-based access control, and Row Level Security policies enforced at the database.

Layer	Responsibility
Cache **Redis**	Active order state, session data, frequently accessed menus, rate limiting, and cached analytics.
Storage **Supabase Storage**	Dish imagery, brand assets, and staff photos.
Email **Resend**	Branded transactional and bulk campaign delivery with tracked engagement events.

4. Technology Stack

Layer	Technology	Role
Frontend	Next.js (React)	SSR menu, dashboards, website, routing, PWA-friendly
UI	Tailwind CSS + shadcn/ui	Consistent branded component library
Backend	FastAPI (Python)	Centralised business logic and API
Data models	SQLModel + Pydantic	Typed ORM models and request/response validation in one layer
Migrations	Alembic	Versioned, reviewable schema changes
Database	Supabase PostgreSQL	Relational system of record
Realtime	Supabase Realtime	Live order, menu, and dashboard updates
Authentication	Supabase Auth	Staff login, RBAC, Row Level Security
Storage	Supabase Storage	Dish images and brand assets
Cache	Redis	Active queues, sessions, cached analytics, rate limiting
Email / Marketing	Resend	Branded campaign and transactional delivery
Deployment	Docker	Containerised, portable services
Reverse proxy	Nginx	Routing and TLS termination
CI/CD	GitHub Actions	Automated build and deploy
Monitoring	Sentry + Grafana	Error tracking and metrics
Hosting	Vercel (frontend) + VPS (backend)	Flexible; avoids single-vendor lock-in

Using SQLModel with Pydantic means the same typed model definitions serve the database schema, the API contract, and validation — reducing drift between layers — while Alembic keeps every schema change versioned and reversible.

5. Roles, Permissions & Access Control

5.1 Configurable Permission Model

Roles are **not hard-coded**. The platform ships with a permissions engine that a **Super Admin** controls directly from the system, so the role structure can change as Sweet1ne's operation changes without a development request.

The Super Admin can:

- Create new roles and name them to match how the business actually works
- Define exactly what each role can see and do, permission by permission — menu editing, pricing, order handling, analytics access, campaign controls, user management, and branch scope
- Assign, promote, demote, or suspend individual users
- Scope a role to a single branch or across both branches

- Retire a role that is no longer relevant

This avoids a fixed structure that becomes irrelevant six months in. Permissions are enforced server-side in FastAPI and again at the database through Supabase Row Level Security — never in the UI alone.

5.2 Suggested Starting Roles

These are provided as ready-made presets at setup. Each one is fully editable, and none of them is fixed.

Suggested Role	Typical Responsibilities
Customer	Scans table QR, browses menu, sets dietary preference, builds and submits an order, tracks status live — no login required.
Floor Staff	Views incoming orders for their branch, manages service, marks orders served.
Branch Admin	Manages branch menu content, availability and pricing; views branch analytics.
Main Admin	Oversight across branches: menus, QR codes, staff accounts, analytics, campaign controls, reporting.
Technical Support	Scoped maintenance access, time-limited.
Super Admin	Full control, including creating and editing all other roles and their permissions.

Customers are never authenticated. Each customer session is scoped to its own table through a token embedded in the QR link, which prevents any cross-table interference.

6. Core Workflows

6.1 QR Menu & Order Workflow (V1)

1. Customer scans the table QR code → the digital menu loads, with branch and table embedded in the link.
2. Customer optionally sets a dietary preference → matching dishes are highlighted and conflicting ones filtered out.
3. Customer adds items and builds an order.
4. The order is submitted to the branch staff dashboard.
5. Staff process the order through its lifecycle; status updates propagate live via Supabase Realtime.
6. The customer's tracking page updates in real time at each stage.
7. Completed orders feed the analytics engine.

> **Note.** Payment processing is out of scope for this programme. Order capture, submission and tracking are delivered; payment capture would be a future Change Request.

6.2 Dietary & Allergen Filtering

Every menu item carries structured tags: allergens (nuts, dairy, gluten, shellfish and others), dietary category (vegan, vegetarian, low-sodium and others), and spice level. When a customer selects a preference, a deterministic rule-based filter removes any item containing a flagged allergen or conflicting with the stated preference, and highlights the remaining suitable dishes.

The rules are custom-built and fully transparent — every result can be traced to a specific tag on a specific dish. Suggestions are clearly labelled as guidance, with a visible note that guests with serious conditions should confirm ingredients with staff.

6.3 Menu Management Workflow

1. An authorised user edits dish content, price, availability, or tags.
2. Changes are written to Supabase PostgreSQL via FastAPI.
3. Supabase Realtime propagates the update so customer menus and staff dashboards reflect it immediately.
4. Content is maintained once and rendered consistently across the QR menu and, in V2, the website.

6.4 Lead Capture & Marketing Workflow (V2)

1. A visitor interacts with the website or a QR journey.
2. A lead is captured with consent, source, and branch tags into the central contact database.
3. An authorised user builds a campaign from editable branded templates with personalisation.
4. Resend delivers the campaign, only within the client's documented approval.
5. Opens, clicks, and attributed events feed back into the analytics layer.

6.5 Insight & Recommendation Workflow (V2)

The **Collect → Analyse → Critique → Alert → Act** cycle:

Stage	What happens
Collect	The platform gathers scans, views, searches, orders, website, and email data.

Stage	What happens
Analyse	Each metric is compared against a benchmark — last week, the branch average, or a target.
Critique	Patterns that cross a calibrated threshold are identified.
Alert	Only meaningful results are surfaced, never a wall of charts.
Act	Each alert resolves into a specific, low-effort next step.

Examples of the rules in practice:

- A dish viewed often but rarely ordered → suggestion to revisit its price, description, or photo.
- A dish not viewed in two weeks → suggestion to feature or remove it.
- Rising searches for a term with no matching dish (for example “vegan”) → suggestion to consider adding one.
- A branch-level drop in scan rate → prompt to check QR placement or table visibility.

The engine is built as an explicit, custom rule set rather than a black box. Every alert states which metric moved, what it was compared against, and which threshold it crossed, so the reasoning is always visible and auditable. Thresholds are calibrated against real pilot data so recommendations are specific to Sweet1ne rather than generic, and they remain adjustable after handover.

The insight engine provides decision support only; the client remains responsible for reviewing and acting on recommendations.

7. Data Model

Core relational entities in Supabase PostgreSQL, defined as SQLModel classes with Pydantic validation:

Entity	Key Fields
branches	id, name, location, settings
tables	id, branch_id, qr_token, status, capacity
menu_items	id, branch_id, name, price, category, image_url, allergens[], dietary_tags[], available
orders	id, branch_id, table_id, status, created_at
order_items	id, order_id, menu_item_id, quantity, notes
users	id, name, email, active, created_at
roles	id, name, description, branch_scope, editable
permissions	id, role_id, capability, allowed
analytics_events	id, branch_id, type (scan / view / search / order), payload, created_at
leads	id, name, contact, consent, source, branch_tag, created_at
campaigns	id, name, template, audience, status, sent_at
campaign_events	id, campaign_id, lead_id, event (open / click), created_at
insight_rules	id, name, metric, comparison, threshold, active
insights	id, branch_id, rule_id, finding, recommendation, created_at

The relational structure enables JOIN-based reporting — revenue trends, top items, branch comparisons, campaign performance — which a document store handles poorly.

8. Security & Data Protection

- Staff authenticate through Supabase Auth. Role checks are enforced server-side in FastAPI and again at the database through Row Level Security policies — not in the UI alone.
- Customers are unauthenticated and scoped to their own table via the QR session token.
- All traffic is served over HTTPS.
- **Sweet1ne is the data Controller** and determines lawful use of customer personal data; GlobalSolutions acts as **Processor** on the client's documented instructions.
- No live customer personal data is made available to developers until the Data Processing Agreement is signed, least-privilege access is approved, and any required transfer safeguard is in place. Development uses synthetic or controlled data until then.
- Consent, source, and retention are captured at the point of lead collection; marketing is sent only within documented approval.
- Sensitive credentials are never stored in repositories. Confidential data and proprietary code are not placed in public or unapproved tools.

9. Delivery Plan

9.1 Plan at a Glance — 18 Weeks Total

Stage	Phase	Focus	Duration	Weeks
V1	1. Setup	Branch, dishes, photography, baseline	1 week	Week 1
V1	2. Build	Menu, ordering, dashboard, analytics	3–4 weeks	Weeks 2–5
V1	3. Test	Real devices, browsers, slow connections	1 week	Week 6
V1	4. Pilot	Live in one branch, controlled comparison	3 weeks	Weeks 7–9
V1	5. Expand	Review evidence, roll out to second branch	1 week	Week 10
V2	6. Discovery & Metric Alignment	Metrics, website content, brand assets, email requirements	1 week	Week 8 (parallel with pilot)
V2	7. Integrations	Google Analytics and Meta connections	2 weeks	Weeks 9–10 (parallel)
V2	8. Website & Lead Capture	New Sweet1ne website with embedded menu; automatic lead collection	5 weeks	Weeks 9–13 (parallel start)
V2	9. Intelligence Engine & Email Marketing	Collect → Analyse → Critique → Alert → Act; branded campaign system	5 weeks	Weeks 11–15
V2	10. Calibration & Shadow Run	Tune thresholds on live V1 data; test campaigns	2 weeks	Weeks 16–17
V2	11. Go-Live & Meeting Replacement	Weekly insight brief replaces the meeting	1 week	Week 18

Key points

- V1 complete and live in both branches by **Week 10**, within the 8–10 week target.
- V2 live at **Week 18** — the full programme lands at 4.5 months, reflecting the two-week extension for the expanded V2 scope.
- By calibration time (Week 16), V1 has been generating live data for 9 weeks — more than enough to cover weekly patterns and day-of-week variation without any dedicated waiting phase.

9.2 Near-Term Detail — Weeks 1 to 6 (V1 to Production)

Week	Focus	Work
Week 1	Setup	Project kickoff; repository and CI/CD scaffolding; Supabase project provisioned; initial schema defined in SQLModel with Alembic migrations; Supabase Auth wired with the permissions engine; branch and table entities; QR token scheme. Client provides pilot branch, starter dishes, photography and baseline sales data.
Week 2	Build — foundation	Menu data model complete; admin menu CRUD; image upload to Supabase Storage; base branded UI (Tailwind + shadcn/ui) using the ivory, white and gold palette.

Week	Focus	Work
Week 3	Build — customer menu	Customer QR menu rendering per branch and table; dish detail views; dietary and allergen tags; search and filtering; rule-based dietary filter.
Week 4	Build — ordering	Order building and cart; order submission to the backend; order and order-item persistence in PostgreSQL; order lifecycle states.
Week 5	Build — dashboard & realtime	Staff dashboard; role and permission enforcement; live order status through Supabase Realtime; real-time menu availability control; per-branch analytics event capture (scans, views, searches, orders).
Week 6	Test	Testing across real devices, browsers and slow connections; performance and hardening; defect fixes. **V1 feature-complete, tested and ready for pilot deployment.**

10. Developer Fees & Payment Authority

10.1 Fee Basis

Fee basis	Milestone
Currency	USD
Total fee / maximum cap	550 USD (developer) + approx. 200 USD (project cost)
Payment method	M-Pesa
Bank / wallet reference	Phone 0711629594
Fees and exchange costs	Developer responsible for receiving fees. Sender responsible for transfer charges.
Invoice / evidence required	Yes — M-Pesa receipt

10.2 Development Phase Payment Schedule

#	Milestone	When	Amount
1	Start of project work	Week 1 — 27 July 2026	115 USD
2	Complete build of V1 to production	Week 6	115 USD
3	Start of V2	Week 10	115 USD
4	Full development and deployment of V2	Week 18	205 USD
	Total developer fee		**550 USD**

10.3 Acceptance & Change Control

- **Review period:** each submitted milestone may be reviewed within 5 Business Days.
- **Rejection:** a rejection should be written and itemised, identifying a material failure against an agreed acceptance criterion.
- **Rectification:** valid material failures are addressed within 10 Business Days and resubmitted.
- **Deemed acceptance:** if no compliant written rejection is received within the review period, the milestone is deemed accepted. Minor defects that do not prevent operational use do not delay acceptance and are addressed in testing or the post-launch support period.
- **Acceptance criteria:** all agreed functionality delivered; no critical defects; source code submitted; production deployment successful; documentation delivered.
- **Revisions:** up to three consolidated design revision rounds are included within scope. A new page type, integration, role structure, workflow, data model, provider, or feature is a Change Request, not a revision.
- **Change approval route:** written approval from Kelvin Kibiru or another authorised GlobalSolutions representative.

11. Deployment, Handover & Support

- Deployment to the agreed production environment: release preparation, environment configuration, launch checks, domain and DNS assistance, and handover guidance.

- On full payment, GlobalSolutions hands over project-specific source code, database access or agreed export, admin and deployment credentials, and technical documentation per the handover register.
- Ongoing server administration, infrastructure monitoring, backups, domain renewal, and 24-hour cover are excluded unless separately contracted.
- **Post-launch support:** 60 calendar days from final go-live, covering reproducible defects, delivered-functionality errors, and deployment issues attributable to the delivered code. New features, redesign, and ongoing content or campaign operation are excluded.

12. Deliverables Summary

1	Customer QR Menu	11	Sweet1ne Website
2	Staff Dashboard	12	Lead Collection System
3	Admin Dashboard	13	Contact Database
4	Branch Management	14	Email Campaign Module
5	Authentication System	15	Insight Engine (Collect → Analyse → Critique → Alert → Act)
6	Configurable Roles & Permissions Engine	16	Weekly Insight Brief
7	Menu Management	17	Deployment & Production Build
8	Image Management	18	Source Code
9	Customer Ordering	19	Technical Documentation
10	Analytics Dashboard	20	API Documentation

13. Brand & Visual Direction

The platform's UI — customer menu, staff dashboard, website, and insight layer — carries Sweet1ne's existing brand identity: a warm gold tone (**#D4AF37**) against an ivory and white base, reflecting the “soul meets food” positioning. The digital experience should read as a natural extension of the Sweet1ne brand rather than a generic third-party tool.

14. Assumptions & Risks

Item	Note
Timely client dependencies	The 18-week target assumes content, access, and approvals arrive on schedule; delays extend the programme.
Third-party services	Email deliverability, analytics, and attribution depend on provider capabilities, consent, and browser settings, and are not guaranteed to be complete.
Insight accuracy	Thresholds are calibrated on pilot data; accuracy improves as more real data accumulates.
Payments excluded	Order capture is delivered; payment processing would be a future Change Request.
Outcomes not guaranteed	No specific revenue, scan rate, conversion, or list-growth figure is guaranteed.

Prepared by SpectreTech Limited — Reliable Tech Solutions
For Developer Stephen Nyansoho Machera and GlobalSolutions · Project Sweet1ne · July 2026